

Digital Circuits and VHDL – Overview and Examples

Florian Hagemann

Abstract—Here be an abstract...

CONTENTS

I	Introduction	1
II	Digital Circuits	1
II-A	Electronic Switches: Transistors	2
II-B	Boolean Algebra	2
II-C	Logic Gates	2
III	Introduction to VHDL	3
III-A	Hardware Description Languages	3
III-B	Basic Syntax	3
III-C	Blackbox Diagrams and Entities	3
IV	Combinational Circuits	3
IV-A	Basic Logic Gates	3
V	Sequential Circuits	3
VI	Finite State Machines (FSM)	3
VI-A	Moore Machines	3
VI-B	Mealy Machines	3
	References	3

closed, electricity can flow through the LED and it will light up; if it is not closed, no electricity will flow.

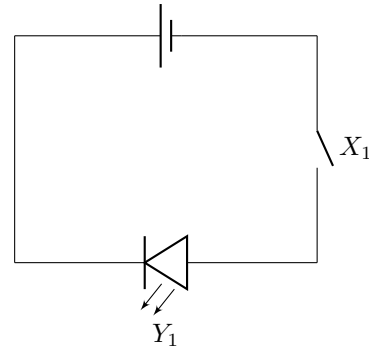


Fig. 1. Simple series circuit with a voltage source, switch, and LED.

We only care for the switch and the output state of the circuit though, not what the power source is or what the output is connected to, so we can simplify our diagram (Fig. 2).

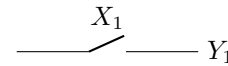


Fig. 2. Simplified diagram of Fig. 1.

So we know if we press X_1 , Y_1 will be set high or to '1'. We can illustrate this using a *truth table* (Tab. I):

TABLE I
TRUTH TABLE FOR FIG. 2

X_1	Y_1
0	0
1	1

With a truth table we can read what our output will be for any input. At this point in time this is overkill, but it gets more useful the more switches we add. For example, we can add another switch in series (Fig. 3).

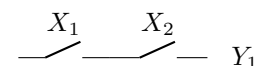


Fig. 3. Two switches in series

Now if none or only one switch is turned on, the output will stay low. But if both switches are turned on, the output will be high. So we can check a condition just using switches

WHAT is a *digital circuit*? To understand this, we need to examine each word separately. A *circuit* is a closed loop that allows electricity to flow. Typically it consists of a power source and some device (e.g., a motor, LED, etc.) that uses that electricity. A circuit needs to be closed in order for the electricity to flow through it [1]. *Digital* comes from the Latin “*digitalis*” meaning finger or toe. Digital can mean “something to do with digits (fingers or toes)” but can also mean the “property of representing values as discrete, often binary, numbers rather than a continuous spectrum” [2].

A **digital circuit** is a closed loop that electricity can flow through that uses discrete values. Most often these values are '1' (on, high, or true) and '0' (off, low, or false)¹. This system of using two values is called *binary*.

We can now look at a very basic example (Fig. 1) of what we can do by defining binary values. When the switch is

¹Thanks to ...

¹You can use more than two values but binary is most common. See https://en.wikipedia.org/wiki/Ternary_computer

TABLE II
TRUTH TABLE FOR FIG. 3

X_1	X_2	Y_1
0	0	0
1	0	0
0	1	0
1	1	1

and electricity. But we might also notice that if we want to do something more complex, we will need more and more switches which we have to flip manually.

A. Electronic Switches: Transistors

Luckily there is an invention that solves the issue of us having to flip switches by hand. The **transistor** is a semiconductor device used (among other uses) to switch electrical signals². Transistors usually have three terminals a *source* (or *collector*), *gate* (or *base*), and *drain* (or *emitter*). You might have heard a water analogy used to explain electricity before and we can use it again to understand the transistor. The water flow comes from the source, applying a voltage to the gate opens it, and the water drains through the drain. If we don't open the gate, no water can flow and the drain will be dry. There are many types of transistors, but we can't talk about all of them, but in general they do the same thing.

We can now replace our switches in Figure 3 with transistors, we will look at the circuit as a whole though (Fig. 4). Now we only need to apply a voltage to X_1 and X_2 instead of switching them. While at this scale it will still be the same amount of work for us to switch the voltages, the advantage of this design is that we can now plug the output of these transistors into another transistor to switch that automatically.

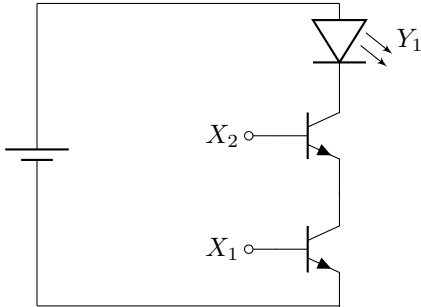


Fig. 4. Simple transistor switch circuit controlling an LED output (Y_1) via input signals X_1 and X_2 .

We represent transistors (here NPN-Transistors) using this symbol (Fig 5).

B. Boolean Algebra

We've been treating the binary system mostly as "on" and "off" up to this point, but in the beginning we also defined '0' as *false* and '1' as *true*. We can use mathematical formula to

²We will not talk about how a transistor works. See <https://en.wikipedia.org/wiki/Transistor> or search the web for more information.

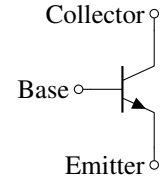


Fig. 5. NPN-Transistor

describe our circuits. For example we can describe the circuit in Figure 3 with the formula

$$X_1 \wedge X_2 = Y_1$$

. The $\wedge$ symbol means logical AND with the upside down $\vee$ meaning logical OR. How they operate is in the name:

- For $\wedge$ or $\cdot$ (AND): *Input One and Input Two* need to be true for the output to be true.
- For $\vee$ or $+$ (OR): *Input One or Input Two* needs to be true for the output to be true.

We can also look at the truth tables of both logical operations (remember that we represent *false* with 0 and *true* with 1).

TABLE III
TRUTH TABLES FOR AND AND OR OPERATIONS

AND ($X_1 \wedge X_2$)			OR ($X_1 \vee X_2$)		
X_1	X_2	Y_1	X_1	X_2	Y_1
0	0	0	0	0	0
1	0	0	1	0	1
0	1	0	0	1	1
1	1	1	1	1	1

Another operator is *negation*, which is written with the $\neg$ symbol, but can also be written like this

$$\neg X = \overline{X}$$

, which you would read as "not X " (Tab. IV).

TABLE IV
TRUTH TABLE FOR NOT

X	$\overline{X}$
0	1
1	0

C. Logic Gates

We want to use these logical operation a lot without drawing a bunch of transistors. That is why there are standard ways of drawing so called **logic gates**. These can be defined as "a device that performs a Boolean function, a logical operation performed on one or more binary inputs that produces a single binary output" [3]. We have already unknowingly used an AND gate in Figure 4. We can redraw our diagram to use a logic gate notation to save ourselves time and space, while making what the circuit does more clear (Fig. 6)³.

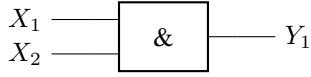


Fig. 6. AND gate with European notation.

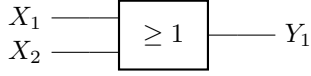


Fig. 7. OR gate with European notation.

These logic gates are very simple and can be used to build a lot of other useful circuits. For example we can build a circuit that only outputs a '1' if only one of the inputs is '1' (Fig. 9 and tab. V). This kind of circuit is called an XOR gate (XOR being short for exclusive or).

TABLE V
TRUTH TABLE FOR XOR

X_1	X_2	Y_1
0	0	0
1	0	1
0	1	1
1	1	0

The main point of logic gates is that we can abstract the electronical components away to only focus on the logic.

III. INTRODUCTION TO VHDL

A. Hardware Description Languages

B. Basic Syntax

C. Blackbox Diagrams and Entities

IV. COMBINATIONAL CIRCUITS

A. Basic Logic Gates

V. SEQUENTIAL CIRCUITS

VI. FINITE STATE MACHINES (FSM)

A. Moore Machines

B. Mealy Machines

REFERENCES

- [1] Wiktionary contributors. circuit. Wiktionary, the free dictionary. [Online]. Available: <https://en.wiktionary.org/wiki/circuit>
- [2] ——. digital. Wiktionary, the free dictionary. [Online]. Available: <https://en.wiktionary.org/wiki/digital>
- [3] Wikipedia contributors, "Logic gate — Wikipedia, the free encyclopedia," 2026, [Online; accessed 4-October-2026]. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Logic_gate&oldid=1377481077

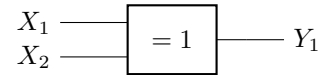


Fig. 10. XOR gate with European notation.

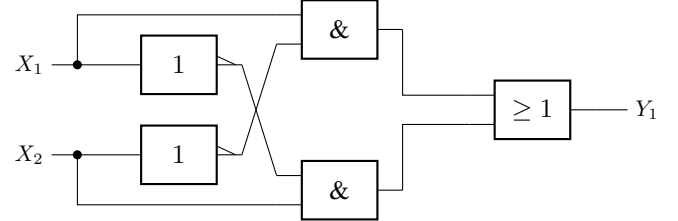


Fig. 9. XOR gate constructed from NOT, AND, and OR gates.

³There are several different notation styles for logic gates. We will use the European notation. See <https://jlcpcb.com/blog/logic-gate-symbols-guide> for an overview of european and ANSI/IEEE equivalents

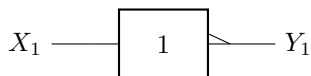


Fig. 8. NOT gate with European notation.